COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	September 2, 2026	**DATE SUBMITTED:**	September 2, 2026
NAME:	Jaime Luis M. Demain	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Seatwork 7.2 **Algorithm Analysis: Best-Case and Worst-Case Time Complexity**

1. PROCEDURE OUTPUT

What procedures did you perform?

1. Finding an Element

```C/C++
for (int i = 0; i < n; i++)
{
    if (arr[i] == target)
    {
        return i;
    }
}
```

Best-Case Time Complexity: O(1)
Worst-Case Time Complexity: O(n)
Explanation: In the best case, the target value will be found at arr[0], which means that the function will return on the first step itself O(1). But in the worst case, the target will either be at the nth index or will not be in the array at all that forces loop to check or all elements O(n).

2. Printing All Elements

```C/C++
for (int i = 0; i < n; i++)
{
    cout << arr[i] << endl;
}
```

Best-Case Time Complexity: O(n)
Worst-Case Time Complexity: O(n)
Explanation: The loop will run *n* times regardless of the contents of the array, requiring linear time in all cases.

3. Nested Loops

```
C/C++
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        cout << i << " " << j << endl;
    }
}
```

Best-Case: O(n^2)

Worst-Case: O(n^2)

Explanation: For every iteration of the outer loop for *n* times, the inner loop executes *n* times. This will result in *n* x *n* = n^2 operations, independent of the conditions in the input data.

4. Conditional Processing

```
C/C++
if (n > 100)
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i];
    }
}
else
{
    cout << "Small input";
}
```

Best-Case Time Complexity: O(1)
Worst-Case Time Complexity: O(n)
Explanation: The best case scenario happens when $n \leq 100$ because here, the else statement runs in constant time, O(1). The worst-case scenario happens when $n > 100$ and the for-loop runs *n* times, O(n).

5. Fixed Inner Loop

```
C/C++
for (int i = 0; i < n; i++)
{
```

```
    for (int j = 0; j < 10; j++)
    {
        cout << i << j;
    }
}
```

Best-Case Time Complexity: O(n)
Worst-Case Time Complexity: O(n)
Explanation: Inner loop executes for a fixed number of times (10). This is a constant O(1). The outer loop will run for *n* number of times. The total number of executions will be 10n and this equals O(n).

6. Two Consecutive Loops

```
C/C++
for (int i = 0; i < n; i++)
{
    cout << arr[i];
}

for (int j = 0; j < n; j++)
{
    cout << arr[j] * 2;
}
```

Best-Case Time Complexity: O(n)
Worst-Case Time Complexity: O(n)
Explanation: Loops are sequential but not nested. The total number of operations is $n + n = 2n$. In Big O notation, coefficients are dropped, leading to O(n) in both cases.

7. Dividing by Two

```
C/C++
int x = n;

while (x > 1)
{
    x = x / 2;
}
```

Best-Case Time Complexity: O(log *n*)
Worst-Case Time Complexity: O(log *n*)
Explanation:

In each step, x is divided by 2. The number of necessary iterations needed to reach 1 i $n/2^k = 1 \Rightarrow k = \log_2 n$, which leads to logarithmic time complexity.

8. Nested Loop With Different Limits

```
C/C++
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < i; j++)
    {
        cout << i << j;
    }
}
```

Best-Case: O(n^2)

Worst-Case: O(n^2)

Explanation: The number of iterations in the inner loop is $0, 1, 2, \ldots, n-1$. The total number of iterations is $\frac{n(n-1)}{2} = \frac{n^2 - n}{2}$. This time complexity is determined by the n^2coefficient, thus the time complexity is quadratic in all cases.

9. Early Termination

```
C/C++
for (int i = 0; i < n; i++)
{
    if (arr[i] == target)
    {
        cout << "Found";
        break;
    }
}
```

Best-Case Time Complexity: O(1)
Worst-Case Time Complexity: O(n)
Explanation: The best case happens when arr[0] == target, the function encounters break statement at the first iteration ($\mathcal{O}(1)$). The worst case takes place if the target is not found, meaning that the function iterates through all n elements ($\mathcal{O}(n)$).

10. Two-Level Search

```C/C++
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        if (arr[i] == target)
        {
            break;
        }
    }
}
```

Best-Case Time Complexity: O(n)
Worst-Case Time Complexity: O(n^2)
Explanation: For the best case, that is arr[0] == target, the inner loop exits on j=0 every time, which takes 1 x *n* = *n* operations. For the worst case, when the target is not in the array, the inner loop runs completely n times for each of the n outer loops (*n* x *n* = n^2)

11. Linear Loop With Logarithmic Inner Work

```None
for (int i = 0; i < n; i++)
{
    int x = n;

    while (x > 1)
    {
        x = x / 2;
    }
}
```

Best-Case Time Complexity: O(*n* log *n*)
Worst-Case Time Complexity: O(*n* log *n*)
Explanation: Outer Loop performes n times. In each iteration of outer loop, the inner while loop iterates $\log_2 n$ times to divide the value of x by 2. Total complexity of this algorithm will be: $n \times \log n = \mathcal{O}(n \log n)$.

12. Conditional Nested Loop

```
None
if (arr[0] == target)
{
    cout << "Found";
}
else
{
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cout << arr[i] + arr[j];
        }
    }
}
```

Best-Case Time Complexity: O(1)
Worst-Case Time Complexity: O(n^2)
Explanation: In best case scenario, arr[0]==target is true, which takes constant time, i.e., $\mathcal{O}(1)$. In worst case scenario, arr[0]!=target is true, where the program enters into else block that has two loops nested each of size n, and hence time complexity will be $\mathcal{O}(n^2)$.

13. Finding the Maximum

```
None
int max = arr[0];

for (int i = 1; i < n; i++)
{
    if (arr[i] > max)
    {
        max = arr[i];
    }
}
```

Best-Case Time Complexity: O(n)
Worst-Case Time Complexity: O(n)
Explanation: The finding of the maximum in an unsorted list involves the evaluation of each element from the first position to $n-1$. It does not matter if the condition of arr[i] > max is true or false, the time complexity will still be $\mathcal{O}(n)$.

14. Three Nested Loops

```
None
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        for (int k = 0; k < n; k++)
        {
            cout << i + j + k;
        }
    }
}
```

Best-Case: O(n^3)

Worst-Case: O(n^3)

Explanation: Three nested loops, all running n times, will run the operation $n \times n \times n = n^3$ times.

15. Comprehensive Analysis

```
None
for (int i = 0; i < n; i++)
```

Best-Case Time Complexity: O(1)
Worst-Case Time Complexity: O(*n* log *n*)
Explanation: In the best case where the array element at index 0 is equal to the target element, the function terminates after just one comparison in constant $\mathcal{O}(1)$ time. Worst case happens when the target is not found, leading to the inner loop $\mathcal{O}(\log n)$ running n times.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts, grammar correction, and text formatting**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

Extent of Use: **I used AI to fully understand how some items worked to fully analyze its time complexity. It enabled me to see the picture of its concepts behind it. Also, I used AI to fix my errors in grammar with a brief explanation. Lastly, I used AI for some text formats here as you can see in some items.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.